DEPLOYING A THREE-TIER APPLICATION IN KUBERNETES

PROJECT_URL https://github.com/mayurpandit25/EasyCRUD.git

1. CREATE RDS DATABASE

1. username: admin
2. password: admin123
3. database: student_db
4. Public Access: Yes
5. Sg: 3306 (MYSQL/Aurora)

2. CLONE PROJECT

1. git clone https://github.com/mayurpandit25/EasyCRUD.git
2. cd EasyCRUD

3. BUILD BACKEND

1. cd backend/src/main/resources/
2. vim application.properties
    1. username: admin
    2. password: admin123
    3. database_endpoint: <endpoint>
3. docker build -t backend .
4. docker tag backend mayurpandit25/cbz-backend:latest
5. docker push mayurpandit25/cbz-backend:latest

6. vim backend-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  labels:
    app: backend
spec:
  replicas: 2
  selector:
    matchLabels:
      app: backend
  template:
```

```
  metadata:
    labels:
      app: backend
  spec:
    containers:
      - name: backend
        image: mayurpandit25/cbz-backend:latest
        ports:
          - containerPort: 8080
```

7.vim backend-svc.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: backend-svc
spec:
  type: NodePort
  selector:
    app: backend
  ports:
   - port: 8080
     targetPort: 8080
```

8.kubectl apply -f backend-deployment.yaml
9.kubectl apply -f backend-svc.yaml
10.kubectl get pods
11.kubectl get svc -o wide
12.kubectl get nodes -o wide

13.curl <node-ip>:<node-port>/api
14.http://13.204.225.64:31082/api ----------> on browser

4. BUILD FRONTEND

1. cd frontend
2. vim .env
   VITE_API_URL=http://13.204.225.64:31082/api
3. docker build -t frontend .

4. docker tag frontend mayurpandit25/cbz-frontend:latest
5. docker login
6. docker push mayurpandit25/cbz-frontend:latest

7. vim frontend-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
  labels:
    app: frontend
spec:
  replicas: 3
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
    spec:
      containers:
        - name: frontend
          image: mayurpandit25/cbz-frontend:latest
          ports:
            - containerPort: 80
```

8. vim frontend-svc.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: frontend-svc
spec:
  type: LoadBalancer
  selector:
    app: frontend
  ports:
```

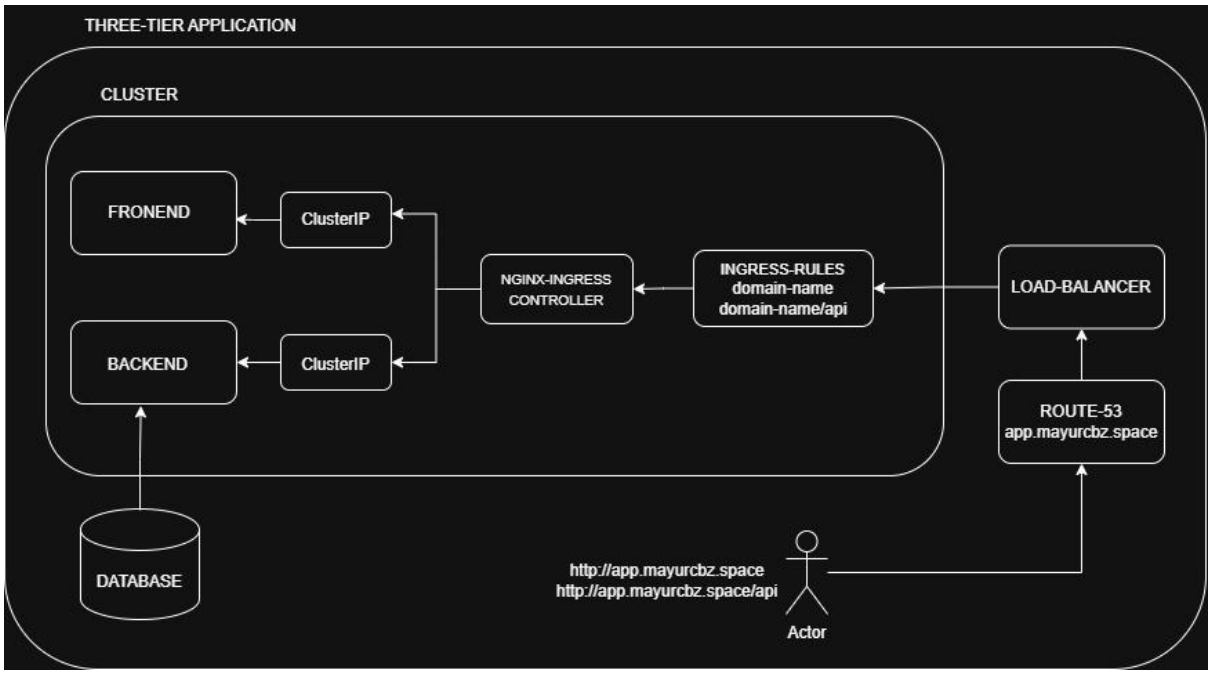
- port: 80
  targetPort: 80

9. kubectl apply -f frontend-deployment.yaml
10. kubectl apply -f frontend-svc.yaml
11. kubectl get pods
12. kubectl get svc -o wide

5. COPY LOAD-BALANCER DNS

1. <frontend-svc-xxxxx.ap-south-1.elb.amazonaws.com> -----> hit on web
2. check the security-group --> 80

DEPLOYING A THREE-TIER APPLICATION IN EKS WITH NGINX-INGRESS + DOMAIN



PROJECT_URL https://github.com/mayurpandit25/EasyCRUD.git

1. CREATE RDS DATABASE

1. username: admin
2. password: admin123
3. database: student_db
4. Public Access: Yes
5. Sg: 3306 (MYSQL/Aurora)

2. CLONE PROJECT

1. git clone **https://github.com/mayurpandit25/EasyCRUD.git**
2. cd EasyCRUD

3. BUILD BACKEND

1. cd backend/src/main/resources/
2. vim application.properties
    4. username: admin
    5. password: admin123
    6. database_endpoint: <endpoint>
3. docker build -t backend .
4. docker tag backend mayurpandit25/cbz-backend:latest
5. docker push mayurpandit25/cbz-backend:latest

6.vim backend-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  labels:
    app: backend
spec:
  replicas: 2
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: backend
          image: mayurpandit25/cbz-backend:latest
          ports:
            - containerPort: 8080
```

7.vim backend-svc.yaml

```
apiVersion: v1
kind: Service
metadata:
```

```
  name: backend-svc
spec:
  type: ClusterIP
  selector:
    app: backend
  ports:
   - port: 8080
     targetPort: 8080
```

8.kubectl apply -f backend-deployment.yaml
9.kubectl apply -f backend-svc.yaml
10.kubectl get pods
11.kubectl get svc

4.TEST BACKEND

1. kubectl get svc
2. kubectl port-forward svc/backend-svc 8081:8080 --address=0.0.0.0 &
3. http://<EC2-PUBLIC-IP>:8081/api

5. BUILD FRONTEND

1. cd frontend
2. vim .env
   VITE_API_URL=/api
3. docker build -t frontend .
4. docker tag frontend mayurpandit25/cbz-frontend:latest
5. docker login
6. docker push mayurpandit25/cbz-frontend:latest
7. vim frontend-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
  labels:
    app: frontend
spec:
  replicas: 3
  selector:
    matchLabels:
```

```
    app: frontend
  template:
    metadata:
      labels:
        app: frontend
    spec:
      containers:
        - name: frontend
          image: mayurpandit25/cbz-frontend:latest
          ports:
            - containerPort: 80
```

8. vim frontend-svc.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: frontend-svc
spec:
  type: ClusterIP
  selector:
    app: frontend
  ports:
    - port: 80
      targetPort: 80
```

9. kubectl apply -f frontend-deployment.yaml
10. kubectl apply -f frontend-svc.yaml
11. kubectl get pods
12. kubectl get svc

6. TEST FRONTEND

1. kubectl get svc
2. kubectl port-forward svc/frontend-svc 8080:80 --address=0.0.0.0 &
3. http://<EC2-PUBLIC-IP>:8080

7. INSTALL NGINX-INGRESS CONTROLLER

1. kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/main/deploy/static/provider/aws/deploy.yaml
2. kubectl get pods -n ingress-nginx

3. kubectl get svc -n ingress-nginx
4. vim ingress.yaml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
spec:
  ingressClassName: nginx
  rules:
    - host: app.mayurcbz.space
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: frontend-svc
                port:
                  number: 80
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: backend-svc
                port:
                  number: 8080
```

5. kubectl apply -f ingress.yaml
6. kubectl get ingress

8. CONFIGURE ROUTE53

1. create a hosted zone (mayurcbz.space)
2. click on created-hosted zone (mayurcbz.space)
3. create a record
    1) record-name: app
    2) record-type: A
    3) alias
        a. choose-endpoint: Alias to network load-balancer
        b. choose-region: ap-south-1

    c. choose load-balancer: Network-Load-Balancer
4. create a record

Note:

1. Add these hosted-zone nameservers on the hostinger domain-name

9. OPEN APPLICATION

Your Frontend Application

1. http://app.mayurcbz.space

Your Backend Application

2. http://app.mayurcbz.space/api
